

Project Design Document

BatkaBOT



Course: Embedded systems Semester 3

Version: V2.0

Authors: Simeon Stoilov, Veselin Atanasov, Plamen Tsokov

Contents

Project Design Document	1
Project Overview	3
Project requirements	3
Part 1	3
Part 2	3
Hardware components	3
Hardware Diagram	4
State Diagram	5
UML Class Diagram	6
Class Diagram Explanation	6
Timer	6
UI	7
Motors	7
Ultrasonic	7
Robot (main.cpp)	8
Conclusion and Reflection	8

Project Overview

The goal of this project is to develop a swarm of robots. Those robots will go in random directions until they reach proximity of 10 cm to an object (another robot or a wall) by gradually reducing their speed, afterwards, the robot will turn to a random angle (from -90 to 90 degrees) and then it will continue the same pattern over and over again. The robots will feature an ultra-sonic sensor for distance measuring, two parallax servos which will deliver power to the wheels and a PID system to control the movement of the robots.

Project requirements

The process of delivering the final product will be separated into three parts. In the first phase, we will implement the functionality for the basic movements of the robots. Moving forward, detecting how far the robot is from an object and rotating.

Part 1

FR1 – Robot will move straight forwards until it encounters an obstacle at 10 cm (distance measured by an ultrasonic sensor, further called “obstacle distance”).

FR2 – Robot will change its movement direction randomly (in the range of -90 ... 90 degrees) if it encounters an obstacle (if it is situated at “obstacle distance” from the obstacle).

FR3 – The speed of the robot and the “obstacle distance” can be controlled via a serial command interface.

Part 2

FR4 – Robot will start to apply FCS PID to brake smoothly when its distance to an obstacle is double of the “obstacle distance”.

FR5 – When the robot achieves “obstacle distance” close enough it should stop applying FCS and randomly change its direction as described in FR2.

Hardware components

Hardware components used:

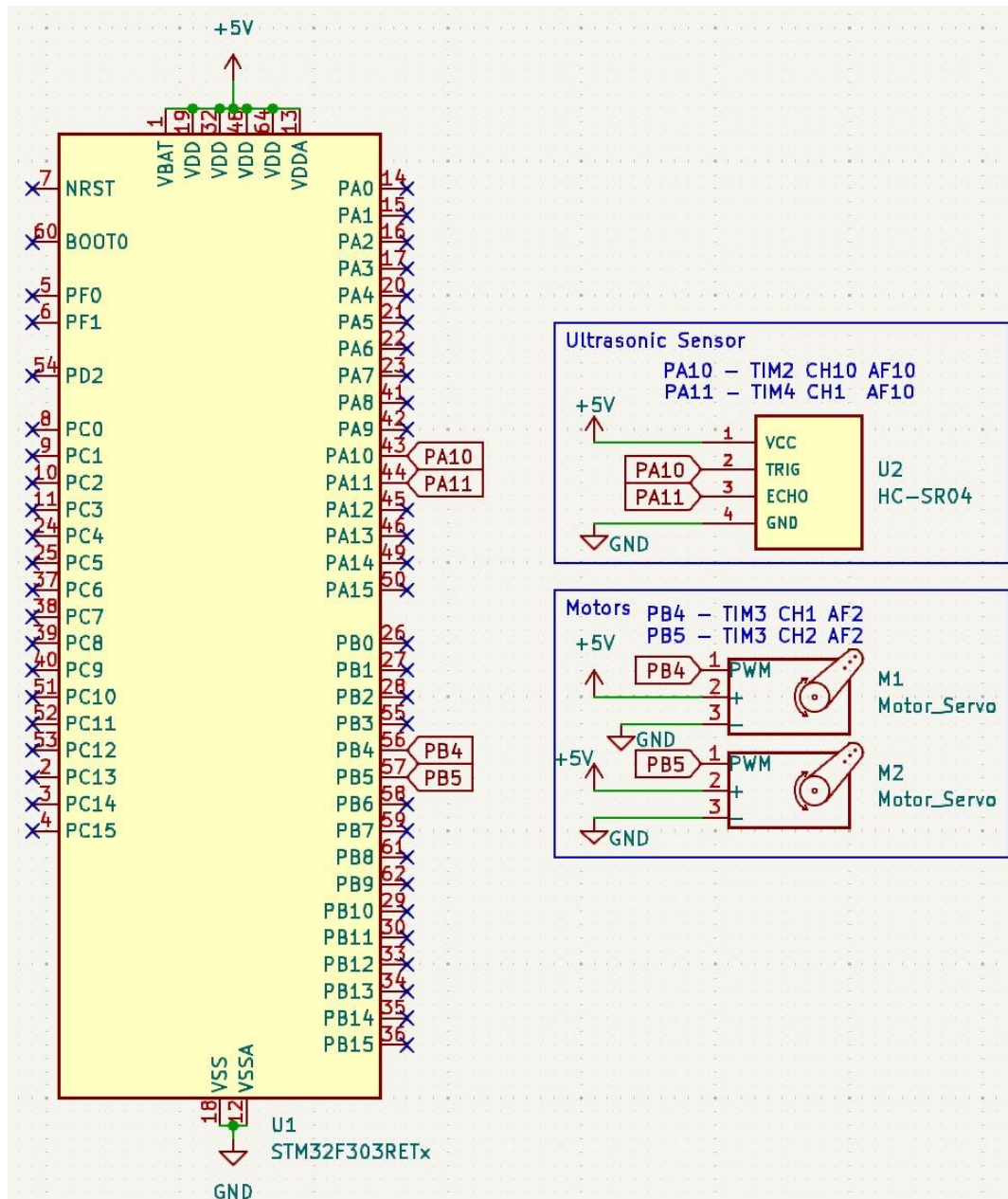
- 3x FHICT Bot
 - 1x HC-SR04
 - 2x Parallax Servo Motor

- 3x NUCLEO F303RE STM32 embedded board

Pins Needed per MCU:

- 2x Parallax Servo Motor – 2 output pins
- 1x HC-SR04 – 1 output, 1 input pins

Hardware Diagram



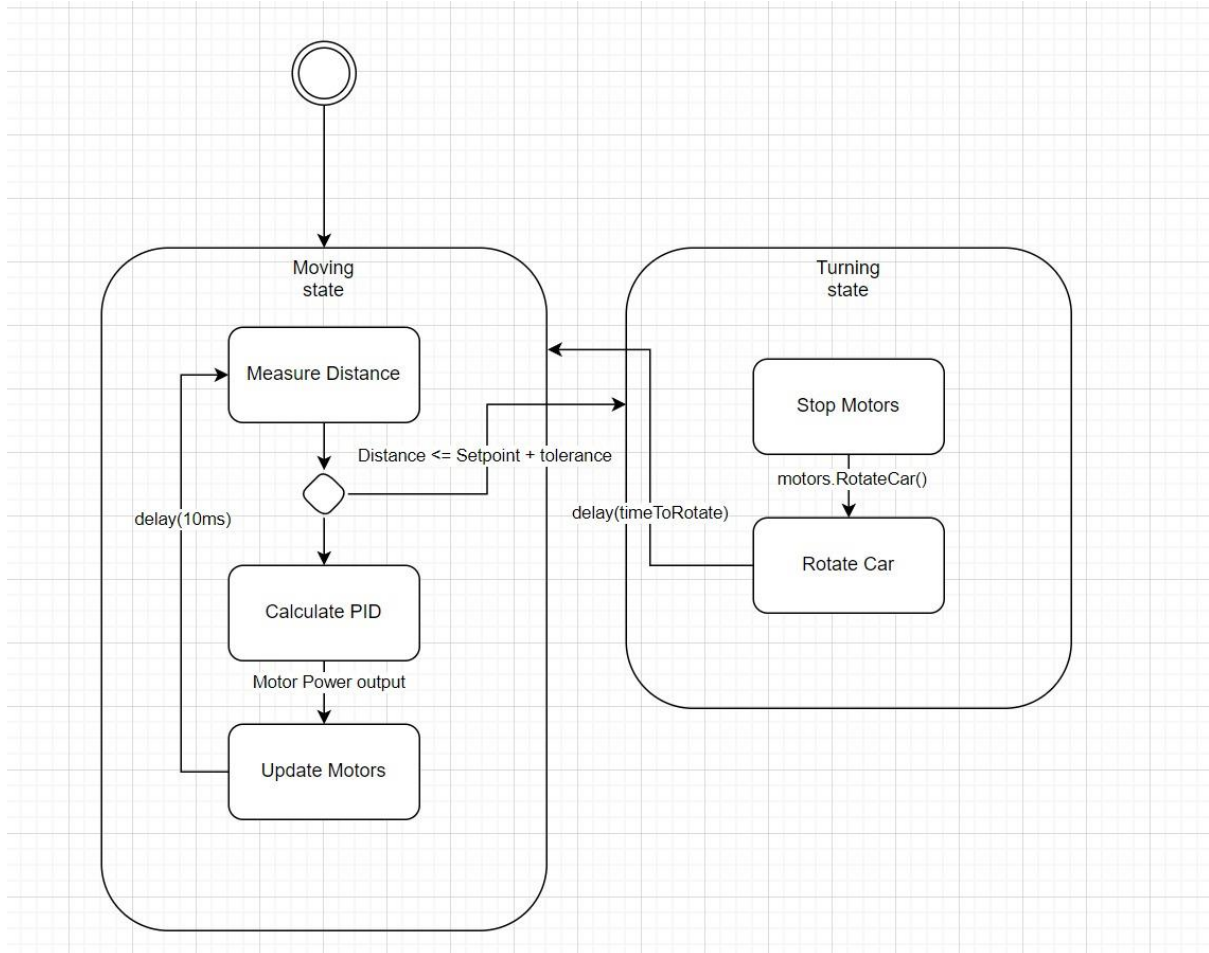
Ultrasonic Sensor pins -

- PA10 as trigger output - TIM2 CH4 – AF10
- PA11 as echo input - TIM4 CH1 - AF10

Parallax 360 Servos -

- PB4 for Servo 1 – TIM3 CH1 AF2
- PB5 for Servo 2 – TIM3 CH2 AF2

State Diagram



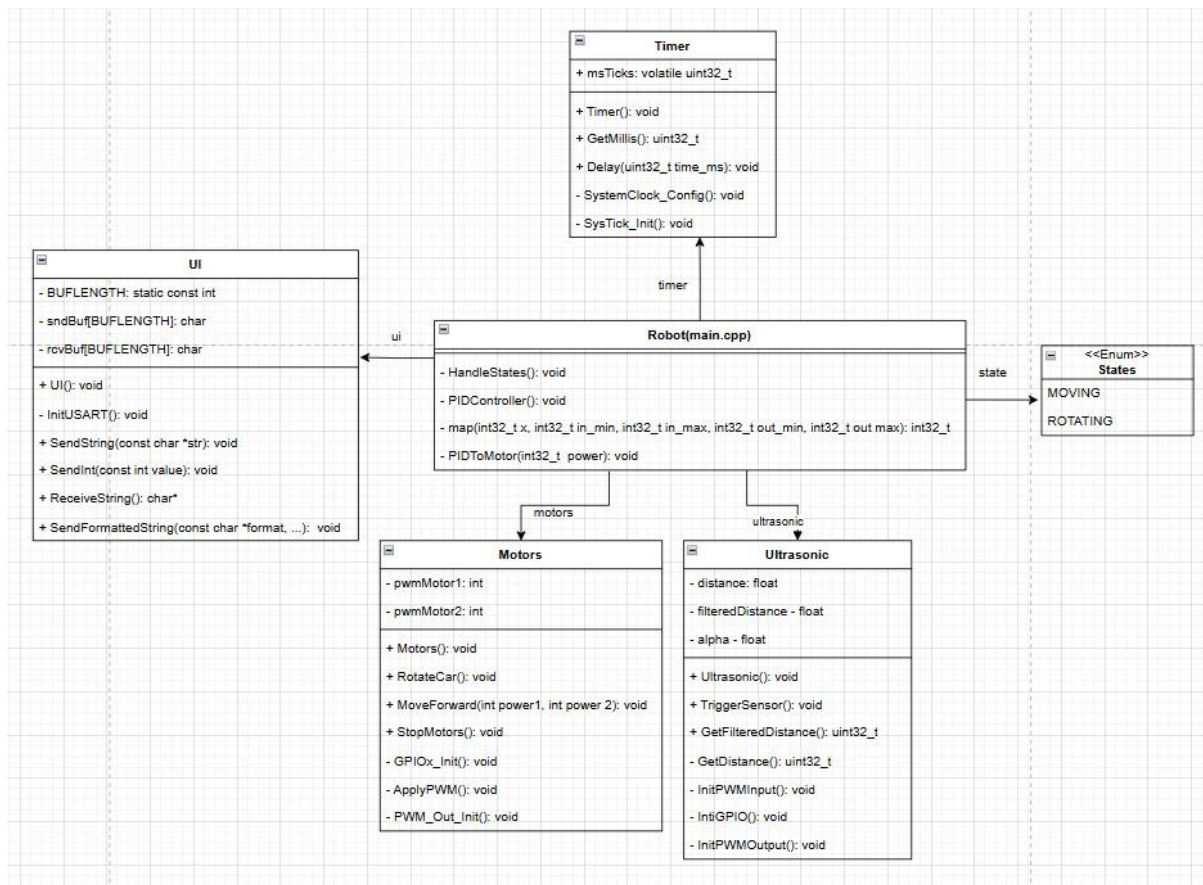
1. Moving State

- **Measure Distance:** Continuously measures the distance from a target object.
- If the measured distance is within the setpoint $\pm$ tolerance, the system transitions to the Turning State.
- **Calculate PID:** If not within range, a PID controller calculates the motor power output based on the distance error.
- **Update Motors:** Adjusts motor power according to the PID output to maintain or approach the desired distance.
- This state iterates every 10ms for smooth updates.

2. Turning State

- **Stop Motors:** Halts the motors to prepare for rotation.
- **Rotate Car:** Executes a pre-defined rotation using `motors.RotateCar()` and waits for a specified time (`timeToRotate`) to complete the turn.
- After turning, the system transitions back to the Moving State for continuous operation.

UML Class Diagram



Class Diagram Explanation

Timer

- **Purpose:** Provides timing functionalities for delays, system ticks, and millisecond counters.
- **Attributes:**
 - `msTicks`: A volatile variable to track milliseconds.
- **Methods:**
 - `Timer()`: Constructor to initialize the timer.
 - `GetMillis()`: Returns the elapsed milliseconds.
 - `Delay(uint32_t time_ms)`: Creates a delay for the given time in milliseconds.
 - `SystemClock_Config()` and `SysTick_Init()`: Initialize and configure the system clock and SysTick timer.

UI

- **Purpose:** Handles communication with the user interface through USART.
- **Attributes:**
 - BUF_LENGTH: Buffer size for communication.
 - sndBuf and rcvBuf: Buffers for sending and receiving data.
- **Methods:**
 - InitUSART(): Initializes the USART interface.
 - SendString() and SendFormattedString(): Methods to send data strings.
 - SendInt(): Sends an integer over the USART.
 - ReceiveString(): Receives a string from the user interface.

Motors

- **Purpose:** Controls motor actions like rotation, stopping, and forward movement.
- **Attributes:**
 - pwmMotor1 and pwmMotor2: PWM values for motors.
- **Methods:**
 - RotateCar(): Rotates the robot.
 - MoveForward(int power1, int power2): Moves the robot forward.
 - StopMotors(): Stops the motors.
 - ApplyPWM() and PWM_Out_Init(): Configure and apply PWM to motors.

Ultrasonic

- **Purpose:** Uses an ultrasonic sensor to measure distances.
- **Attributes:**
 - distance and filteredDistance: Raw and filtered distance values.
 - alpha: A filtering factor for distance.
- **Methods:**
 - TriggerSensor(): Sends a trigger pulse to the sensor.
 - GetDistance() and GetFilteredDistance(): Calculate and retrieve the measured distance.
 - InitPWMInput() and InitGPIO(): Initialize the PWM and GPIO functionalities for the sensor.

Robot (main.cpp)

- **Purpose:** Implements the robot's main functionality, coordinating other components.
- **Attributes:**
 - ui, timer, motors, and ultrasonic: Instances of other classes.
 - state: Tracks the robot's state (MOVING, ROTATING).
- **Methods:**
 - HandleStates(): Manages transitions between robot states.
 - PIDController(): Implements a PID controller for motor control.
 - map(): Maps input values from one range to another.
 - PIDToMotor(int power): Maps PID output to motor power.

Conclusion and Reflection

1. Plamen:

Overall, the project was a success. My main part was the PWM control of the motor movement and turning. That part works well and fulfils the project requirements. Fine tuning the PID was a big challenge for as a group. Veselin spent a lot of time getting it to work as good as possible. Participating in the process was a great learning experience and gave me incredibly valuable experience in the field of PID. This project was fun and a great learning experience. If I were to change something about the process it would be to activate the PID upon reaching a certain point instead of having it work the entire time. Almost all learning outcomes were covered in this project, which i really appreciate. I'm satisfied with the process and the results.

2. Veselin:

For me this project was really fun and interesting. It is really nice to see your work come to live and actually do something. I am happy with the result, however I feel like more could have been done. I had issues putting the PID into a separate class, because it had to use some objects as timer, ultrasonic and motors which were already in main. If we had more time, what I would have done is put the PID in its own class and also "tidy up" the code more and also find a way to make the car stop more smoothly while keeping the balance between a clean PID and helper functions. As for my colleagues, I am happy with their work, their dedication and the communication between the team.

3. Simeon:

This project was really interesting, and it proved almost all of the learning outcomes for this course. In the time that we had we were almost able to have it perfect, currently it is fully functional, but it isn't as refined as I hoped. Both of the teachers were satisfied with

it so that lifts a huge load. I am happy with the team, I think since we divided the tasks quite early it was very clear what each of us had to do, and when it came to implementing and calibrating the values for the PID controller all of us came together and worked on it to make it what it is now.